

编译单元	CompUnit	→ [CompUnit] (Decl FuncDef)
声明	Decl	→ ConstDecl VarDecl
常量声明	ConstDecl	→ 'const' BType ConstDef { ',' ConstDef } ';'
基本类型	BType	→ 'int' 'float' TensorType
张量类型	TensorType	→ 'tensor' ('int' 'float')
常数定义	ConstDef	→ Ident { '[' ConstExp ']' } '=' ConstInitVal
常量初值	ConstInitVal	→ ConstExp '{' [ConstInitVal { ',' ConstInitVal }] '}'
变量声明	VarDecl	→ BType VarDef { ',' VarDef } ';'
变量定义	VarDef	→ Ident { '[' ConstExp ']' } Ident { '[' ConstExp ']' } '=' InitVal
变量初值	InitVal	→ Exp '{' [InitVal { ',' InitVal }] '}'
函数定义	FuncDef	→ FuncType Ident '(' [FuncFParams] ')' Block
函数类型	FuncType	→ 'void' 'int' 'float' TensorType
函数形参表	FuncFParams	→ FuncFParam { ',' FuncFParam }
函数形参	FuncFParam	→ BType Ident '[' ']' { '[' Exp ']' }
语句块	Block	→ '{' { BlockItem } '}'
语句块项	BlockItem	→ Decl Stmt
语句	Stmt	→ LVal '=' Exp ';' [Exp] ';' Block 'if' '(' Cond ')' Stmt ['else' Stmt] 'while' '(' Cond ')' Stmt 'break' ';' 'continue' ';' 'return' [Exp] ';'
表达式	Exp	→ AddExp
条件表达式	Cond	→ LOrExp
左值表达式	LVal	→ Ident { '[' Exp ']' }
基本表达式	PrimaryExp	→ '(' Exp ')' LVal Number
数值	Number	→ IntConst FloatConst
一元表达式	UnaryExp	→ PrimaryExp Ident '(' [FuncRParams] ')' UnaryOp UnaryExp
单目运算符	UnaryOp	→ '+' '-' '!' 注: '!'仅出现在条件表达式中
函数实参表	FuncRParams	→ Exp { ',' Exp }
乘除模表达式	MulExp	→ UnaryExp MulExp ('*' '/' '%' '@') UnaryExp
加减表达式	AddExp	→ MulExp AddExp ('+' '-') MulExp
关系表达式	RelExp	→ AddExp RelExp ('<' '>' '<=' '>=') AddExp
相等性表达式	EqExp	→ RelExp EqExp ('==' '!=') RelExp
逻辑与表达式	LAndExp	→ EqExp LAndExp '&&' EqExp
逻辑或表达式	LOrExp	→ LAndExp LOrExp ' ' LAndExp
常量表达式	ConstExp	→ AddExp 注: 使用的 Ident 必须是常量

*对 SysY2022 文法修订的地方用红色进行了标记

补充语义说明：

一、类型系统

TensorType 表示一个多维张量，其元素类型为 int 或 float。
张量的秩由变量声明中的维度个数决定，每个维度大小由对应的 ConstExp 确定（必须为正整数）。
两个张量类型 相同 当且仅当：

- 组成张量的元素类型相同（同为 int 或同为 float）；
- 秩相同；
- 每个维度的大小对应相等。

二、声明与初始化

张量变量可出现在任何允许 BType 出现的位置（全局、局部、函数形参、返回值）。下面声明了不同维度的张量变量。可使用初值列表 { ... } 进行初始化，对张量初始化的语义规则同普通数组。可使用下标运算符 [] 访问张量元素，下标个数等于张量的秩。下标越界行为未定义。

初值列表的嵌套层级必须与张量维数匹配。元素个数应等于对应维度大小（可省略尾部的零初始化，与 C 一致）。全局张量未初始化时的值不做约定（未定义）。示例如下：

```
tensor int a[4] = {1, 2};           // a[0]=1, a[1]=2
tensor int b[2][3] = {{1,2,3}, {4,5,6}};
tensor float c[2][2] = {1.0, 2.0, 3.0, 4.0}; //自动按行填充
```

三、算术运算

以下运算符扩展支持张量操作数，必须遵循逐元素运算和标量提升规则。

3.1 一元运算符

- +（正号）：操作数为张量时，结果为同形状张量，每个元素取正（无实际效果）。
- （负号）：操作数为张量时，结果为同形状张量，每个元素取负。

3.2 二元算术运算符

运算符：+、-、*、/、%（% 仅当基类型为 int 时有效）。

操作数组合与规则：

左操作数	右操作数	规则
标量	标量	原有标量运算
张量	标量	标量提升：标量隐式转换为与张量同形状的张量，每个元素等于该标量值，然后执行逐元素运算。结果类型为张量。
标量	张量	同上（标量提升）
张量 A	张量 B	+、-、*、/、%操作的两个张量形状应相同（秩相同且各维度大小相等），执行逐元素运算，结果张量形状不变。*为逐元素乘法（Hadamard 积）。 @运算符用于执行线性代数中的矩阵乘法。若左操作数形状为 M×N，右操作数为 N×P，则结果形状为 M×P。

类型检查：张量的基类型必须与标量类型一致（如 int 张量不能与 float 标量混合）。

两个张量运算时基类型必须相同。

示例：

```
tensor int a[4] = {1,2,3,4};
tensor int b[4] = {5,6,7,8};
tensor int c[4] = a + b * 2 + 10;
tensor int d[4] = a * b;
tensor int e[4] = a % 3;
tensor int f[4] = b % a;
tensor int g[4] = b / a;
// b*2 → 标量2提升为 {2,2,2,2}，逐元素乘 → {10,12,14,16}
// a + ... → {11,14,17,20}
// +10 → 标量10提升，逐元素加 c → {21,24,27,30}
// d → {5,12,21,32}
// e → {1,2,0,1}
// f → {0,0,1,0}
// g → {5,3,2,2}
tensor int A[2][3] = {{1,2,3},{4,5,6}};
tensor int B[3][2] = {{7,8},{9,10},{11,12}};
tensor int C[2][2] = A @ B; // 结果矩阵 2x2
// C[0][0] = 1*7 + 2*9 + 3*11 = 58
// C[0][1] = 1*8 + 2*10 + 3*12 = 64
// ...
```

3.3 关系与逻辑运算

关系运算 (<, >, <=, >=, ==, !=) 不支持张量操作数（视为类型错误）。

逻辑与 (&&)、逻辑或 (||)、逻辑非 (!) 同样不支持张量。

3.4 赋值

张量变量之间可直接赋值，要求赋值号两侧张量类型完全相同（形状和基类型一致）。

不支持张量直接赋给标量变量。

暂不支持标量直接赋给张量变量（张量的值需通过下标逐个赋值或使用初值列表构造）。

3.5 函数参数与返回值

张量作为形参时，第一个维度必须留空（写为 []），后续维度必须指定大小。形参写法与普通数组形参一致。示例：

```
void f(tensor int a[], tensor int b[][10]);
// a: 一维张量，长度由实参决定；b: 二维张量，第二维固定为10，第一维由实参决定。
```

形参中的张量类型实际上是“指向张量的指针”，退化为数组指针语义（与 C 一致）。调用时，实参张量的形状必须与形参匹配。

函数可返回张量类型。示例：

```
tensor int add_vec(tensor int x[], tensor int y[], int len) {
    tensor int res[10][10];
    .....
    return res; // 实际中，可返回固定大小张量，或通过形参传入结果张量。
}
```

为简化实现，测评用例会限制返回张量的尺寸在编译时已知。